

## IT / Niftyブログをgithubへ移行する方法 ▾

以前 nifty でブログを書いていた。その後、そのバックアップを取り、niftyを退会しました。その過去のブログを別の形で公開したいのですがどうすればよいですか。

いくつか確認させてください。

バックアップデータの形式と、公開方法の希望を教えてください。  
回答をお待ちしています。

Q: バックアップデータの形式は？

A: HTML形式

Q: 希望する公開方法は？

A: まだ決めていない

わかりました。HTMLバックアップがあるなら、選択肢はいくつかあります。

---

## 主な公開方法の比較

### 1. GitHub Pages（無料・静的サイト）

- GitHubのリポジトリにHTMLをそのまま置くだけで公開可能
- 独自ドメイン設定も可能
- Linux + gitで完結するので相性が良い
- 向き：技術的に慣れている人

### 2. Blogger / はてなブログ等への移行

- HTMLを記事ごとに手動でインポート or コピペ
- 作業量が多いが、ブログらしいUIになる
- 向き：コメント機能や読者登録が欲しい場合

### 3. 自宅サーバー（zama）での公開

- nginx か Apache を入れてHTMLをそのまま配信
- ただし自宅回線での外部公開はセキュリティ・帯域の考慮が必要
- 向き：外部公開せず身内だけに見せる場合は適している

4. Netlify / Cloudflare Pages（無料・静的ホスティング）

- GitHubと連携してHTMLをホスティング
- GitHub Pagesより高機能（リダイレクト設定など）
- 向き：手軽に本格的なサイトを作りたい場合

判断のポイント

| 考慮点          | 推奨                       |
|--------------|--------------------------|
| 手間をかけたくない    | GitHub Pages または Netlify |
| ブログとして再公開したい | はてなブログ                   |
| 外部に公開しなくてよい  | zama + nginx             |
| 独自ドメインを使いたい  | GitHub Pages / Netlify   |

バックアップのHTMLは、記事ごとに個別ファイルに分かれていますか？それとも一括のファイルですか？構成によって作業量が変わります。

以前のniftyのような商用ブログサイトは使いたくないのです。

なるほど。では実質的な選択肢は3つに絞られます。

絞り込んだ選択肢

1. GitHub Pages（最もお勧め）

- 無料・広告なし・商用サービスへの依存が最小
- HTMLをそのままgit pushするだけで公開
- Linux Mintとの相性も良い
- 独自ドメインも設定可能

2. Netlify / Cloudflare Pages

- GitHub Pages同様、静的HTMLをそのまま公開
- Netlifyは無料枠が十分広い

- ただしアカウント登録が必要（商用サービスではあるが広告は出ない）

### 3. 自宅サーバー（zama）

- 完全自前・外部サービス依存ゼロ
- ただし外部公開するには固定IPまたはDDNSが必要
- セキュリティ管理が自己責任

---

### 確認させてください

- 外部（インターネット）に公開したいのか、身内・自分だけで見られればよいのか、どちらですか？
- 独自ドメインは持っていますか、または取得する意向はありますか？

この2点で最適な選択肢が決まります。

外部に公開したいです。アマチュア無線関連なのですが、他のアマチュア無線局にも役立つ情報が入っていると思いますので。

ありがとうございます。方向性が決まりましたね。

---

### お勧め：GitHub Pages

理由：

- 無料・広告なし・商用ブログへの依存なし
- HTMLバックアップをそのまま使える
- 外部公開可能
- アマチュア無線コミュニティでも利用者多数

---

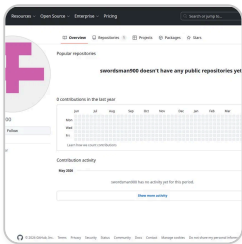
### 大まかな手順

1. GitHubアカウント作成（未取得の場合）
2. リポジトリ作成（例：`JA1XXX.github.io`）
3. HTMLファイルをリポジトリにpush
4. GitHub Pagesを有効化 → `https://JA1XXX.github.io` で公開完了

Linux Mintからgitコマンドで完結します。

事前確認

- GitHubアカウントはすでにお持ちですか？
- バックアップHTMLのディレクトリ構成を確認したいのですが、lsで見た時のファイル構成を教えてくださいませんか？（記事が個別ファイルか、一括ファイルかで作業が変わります）



なぜ GitHub が良いのですか。  
アカウントは持っていますが、一度も使っていません。

GitHubアカウント（swordsman900）をお持ちですね。

GitHub Pagesが良い理由

| 項目       | 内容                                        |
|----------|-------------------------------------------|
| 費用       | 完全無料                                      |
| 広告       | 一切なし                                      |
| 商用依存     | 最小限（Microsoftが運営だが、静的ホスティングは枯れた機能）        |
| 技術要件     | gitコマンドのみ、Linux Mintで完結                   |
| HTMLそのまま | 変換・加工不要                                   |
| URL      | https://swordsman900.github.io/リポジトリ名 で公開 |
| 独自ドメイン   | 後から設定可能                                   |
| 実績       | 技術系・アマチュア無線局の個人サイトで広く使われている               |

## 懸念点も正直に

- GitHubはMicrosoft傘下（2018年～）なので「完全に商用フリー」ではない
- 将来的なサービス変更リスクはゼロではない（ただし現時点では安定）
- gitの基本操作を覚える必要がある（難しくはない）

## 他の選択肢と比べると

Netlify → 同様に良いが、gitと連携前提でやや複雑

自宅サーバー（zama）外部公開 → 固定IPまたはDDNSが必要。セキュリティ管理が自己責任になる

まずバックアップHTMLのファイル構成を確認したいのですが、どのような形で保存されていますか？

excerpt\_from\_previous\_claude\_message.txt

1行

TXT

tar で、こちらにアップロードすればよいですか。

tarのままアップロードしていただければ、こちらで展開して構成を確認できます。

ただし、サイズが大きい場合はアップロードできないことがあります。まず以下を教えてください：

bash

`du -sh アーカイブファイル名.tar*`

のサイズはどのくらいですか？

```
swordsman@mainpc:~/Common/Hamradio$ du -sh backup.tar.gz
1.7M  backup.tar.gz
swordsman@mainpc:~/Common/Hamradio$
```

1.7MBなら問題なくアップロードできます。そのままアップロードしてください。

backup.tar.gz

GZ

ありがとうございます。展開して確認します。

2個のコマンドを実行しました >

構成がよくわかりました。

---

## バックアップの構成

- **ブログ名**： cyberham
- **記事数**： 約241記事（index除く）
- **期間**： 2007～2013年、2023年
- **カテゴリ**： dstar、echoir1p、dextra、asterisk\_app\_rpt、rtpdir、off\_the\_topic など
- **その他**： styles.css、画像ファイル、atom.xml/rss.xml あり

---

## 重要な注意点

`?no_prefetch=1` という名前のファイルが大量にあります。これはNiftyのバックアップツールが誤って作成した重複ファイルで、**公開前に削除が必要です**。

---

## GitHub Pagesへの公開手順（概要）

1. `?no_prefetch=1` ファイルを削除
2. GitHubに `swordsman900.github.io` または `cyberham` というリポジトリを作成
3. ファイルをgit pushする

## 4. GitHub Pagesを有効化

進めてよいですか？まず **no\_prefetchファイルの削除コマンド** から案内します。

まず、github のアカウント名を変えたいです。

GitHubのアカウント名変更は、Web UIから行います。

---

### 手順

1. GitHub.com にログイン
  2. 右上のアイコン → **Settings**
  3. 左メニュー下部 → **Account** をクリック
  4. **Change username** ボタンをクリック
  5. 新しいユーザー名を入力して確認
- 

### 注意点

- 変更後、旧名（swordsman900）のURLは新名にリダイレクトされる（ただし永続保証はない）
  - すでにリポジトリがある場合はリモートURLの更新が必要ですが、今回はまだ何もないので問題なし
  - 希望するユーザー名が既に使われていると取得できない
- 

希望するユーザー名は決まっていますか？（コールサインベースにする方が多いです）

ji1bqw

良い選択ですね。コールサインベースのユーザー名はアマチュア無線関連サイトとして分かりやすいです。

変更できたら教えてください。次のステップ（リポジトリ作成）に進みます。

アカウントに紐付けされているメールアドレスも変更したいです。

それもWeb UIから変更できます。

---

## 手順

1. GitHub.com にログイン
  2. 右上のアイコン → **Settings**
  3. 左メニュー → **Emails**
  4. **Add email address** で新しいメールアドレスを追加
  5. 追加したアドレスに確認メールが届くので認証
  6. 認証後、新アドレスを **Primary** に設定
  7. 旧アドレスを削除（任意）
- 

## 注意点

- 新アドレスの認証が完了するまで旧アドレスは有効なまま
- Primaryに設定してから旧アドレスを削除する順番を守ること

完了したら教えてください。

ユーザー名変更、primary email address 変更、完了しました。

完了おめでとうございます。

次はリポジトリの作成です。

---

## GitHubでリポジトリを作成

1. GitHub.com にログイン
2. 右上の + → **New repository**
3. 以下のように設定：



| 項目                     | 設定値              |
|------------------------|------------------|
| Repository name        | ji1bqw.github.io |
| Visibility             | Public           |
| Initialize with README | チェックしない          |

#### 4. Create repository をクリック

ji1bqw.github.io という名前にすることで、公開URLが https://ji1bqw.github.io になります。

作成できたら教えてください。

<https://github.com/ji1bqw/ji1bqw.github.io>

完了ですね。確認できました。

次はmainpc側の作業です。

## mainpcでの作業

### 1. gitの初期設定（初回のみ）

```
bash

git config --global user.name "ji1bqw"
git config --global user.email "登録したメールアドレス"
```

### 2. 作業ディレクトリの準備

```
bash

cd ~/Common/Hamradio
cp -r "backup.tar.gz" /tmp/ # すでに展開済みなら不要
cd /tmp/blog/Nifty\ Blog\ backup/cyberham
```

まず、`?no_prefetch=1` ファイルの削除から始めます。実行前に確認のため、何個あるか見てみましょう：

```
bash  
find . -name "*no_prefetch*" | wc -l
```

結果を教えてください。

[<https://github.com/ji1bqw.github.io>]  
(<https://github.com/ji1bqw/ji1bqw.github.io>) ではないのですか。

その通りです、失礼しました。

正しくは <https://github.com/ji1bqw/ji1bqw.github.io> です。確認できました。

では次の作業に進みましょう。mainpcで以下を実行してください：

## 1.gitの初期設定（初回のみ）

```
bash  
git config --global user.name "ji1bqw"  
git config --global user.email "登録したメールアドレス"
```

## 2.no\_prefetchファイルの確認

```
bash  
cd ~/Common/Hamradio  
tar xzf backup.tar.gz  
cd "Nifty Blog backup/cyberham"  
find . -name "*no_prefetch*" | wc -l
```

結果を教えてください。

他ユーザーが参照するURLはどうなりますか。

公開URLは以下の2種類です：

## 公開URL

| 用途     | URL                                                                                                                       |
|--------|---------------------------------------------------------------------------------------------------------------------------|
| サイトトップ | <a href="https://ji1bqw.github.io">https://ji1bqw.github.io</a>                                                           |
| 個別記事例  | <a href="https://ji1bqw.github.io/2009/11/d-star-icc-b5a1.html">https://ji1bqw.github.io/2009/11/d-star-icc-b5a1.html</a> |

GitHubのリポジトリページ（<https://github.com/ji1bqw/ji1bqw.github.io>）は管理用なので、一般の閲覧者はそちらを意識する必要はありません。

<https://ji1bqw.github.io/>  
で何もみえません。404です。

まだファイルをアップロードしていないので404が出るのは正常です。

では作業を進めましょう。mainpcで順番に実行してください。

### 1.gitの初期設定（初回のみ）

```
bash

git config --global user.name "ji1bqw"
git config --global user.email "登録したメールアドレス"
```

### 2.バックアップを展開して作業ディレクトリへ移動

```
bash

cd ~/Common/Hamradio
tar xzf backup.tar.gz
cd "Nifty Blog backup/cyberham"
```

### 3.no\_prefetchファイルの数を確認

```
bash

find . -name "*no_prefetch*" | wc -l
```

結果を教えてください。

まずは、シンプルなテストページを公開したいです。

良いアプローチですね。では順番に進めましょう。

## 1. gitの初期設定（初回のみ）

```
bash

git config --global user.name "ji1bqw"
git config --global user.email "登録したメールアドレス"
```

## 2. 作業ディレクトリの作成

```
bash

mkdir ~/github
cd ~/github
git clone https://github.com/ji1bqw/ji1bqw.github.io.git
cd ji1bqw.github.io
```

## 3. テストページの作成

```
bash

cat > index.html << 'EOF'
<!DOCTYPE html>
<html>
<head><meta charset="UTF-8"><title>JI1BQW</title></head>
<body>
<h1>JI1BQW Amateur Radio Station</h1>
<p>Test page</p>
</body>
</html>
EOF
```

## 4. pushする

```
bash

git add index.html
git commit -m "Initial commit"
```

```
git push origin main
```

pushの際にGitHubの認証を求められます。**パスワードではなくPersonal Access Token (PAT)が必要です**。PATはまだ作成していますか？

```
git config --global user.name "ji1bqw"
git config --global user.email "登録したメールアドレス"
```

⇒ これらはローカルに対するコマンドですか。

はい、ローカル（mainpc）に対するコマンドです。

GitHubにpushする際に、コミットに付与される「誰が書いたか」の情報を設定するものです。実行してもGitHubへの通信は一切発生しません。

mainpcではないPC (hp-stream)から実行してしまいました。

問題ありません。mainpcで同じコマンドをもう一度実行するだけです。

```
bash

git config --global user.name "ji1bqw"
git config --global user.email "登録したメールアドレス"
```

`--global` 設定はそのマシンの `~/.gitconfig` に保存されるので、マシンごとに設定が必要です。hp-streamの設定はそのままにしておいても問題ありません。

```
swordsman@mainpc:~/github/ji1bqw.github.io$ git push origin main
Username for 'https://github.com': ji1bqw
Password for 'https://ji1bqw@github.com':
remote: Invalid username or token. Password authentication is not
supported for Git operations.
fatal: Authentication failed for
'https://github.com/ji1bqw/ji1bqw.github.io.git/'
swordsman@mainpc:~/github/ji1bqw.github.io$
```

予想通りです。GitHubはパスワード認証を廃止しているので、Personal Access Token (PAT) が必要です。

## PATの作成手順

1. GitHub.com にログイン
2. 右上アイコン → **Settings**
3. 左メニュー最下部 → **Developer settings**
4. **Personal access tokens** → **Tokens (classic)**
5. **Generate new token** → **Generate new token (classic)**
6. 以下のように設定：

| 項目         | 設定値                 |
|------------|---------------------|
| Note       | mainpc など任意         |
| Expiration | 任意（No expirationも可） |
| Scope      | <b>repo</b> にチェック   |

7. **Generate token** をクリック
8. 表示されたトークン（`ghp_xxxxx...`）を**必ずコピーして保存**（再表示不可）

作成できたら、再度pushします：

```
bash
git push origin main
```

Usernameに `ji1bqw`、Passwordに**PATのトークン文字列**を入力してください。

```
swordsman@mainpc:~/github/ji1bqw.github.io$ git push origin main
Username for 'https://github.com': ji1bqw
Password for 'https://ji1bqw@github.com':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
```

もっと表示

成功です。

ブラウザで確認してください：

**<https://ji1bqw.github.io>**

ただし、初回は反映まで数分かかることがあります。表示されたら教えてください。

テストページが表示されました。

おめでとうございます！GitHub Pagesの公開確認ができました。

---

次のステップは本番のブログファイルのアップロードです。以下の作業が残っています：

1. `?no_prefetch=1` ファイルの削除
2. ブログファイルをリポジトリにコピー
3. `git push`

進めますか？まず以下を実行して確認しましょう：

```
bash
cd ~/Common/Hamradio
tar xzf backup.tar.gz
cd "Nifty Blog backup/cyberham"
find . -name "*no_prefetch*" | wc -l
```

その前に、現在のNiftyからバックアップした旧ブログのフォーマットに問題があります。

まず最初に、バックアップには私が自身で書いた部分だけではなく、Niftyが自動で挿入した飾りの部分も多く含まれているのです。つまり、Niftyフォーマットのページになっているので、そのままgithubで公開はできないのではないかと思います。

もっと表示

良い指摘です。両方とも重要な問題です。

## 1. Nifty固有のHTML装飾について

実際のHTMLを見てみないと判断できません。まず1記事を確認しましょう：

```
bash
cd ~/Common/Hamradio/"Nifty Blog backup"/cyberham/2009/11
cat d-star-icc-b5a1.html
```

内容を貼り付けてもらえますか。Niftyのナビゲーションバー・広告・フッターがどの程度含まれているか確認します。

## 2. 相対パスのリンクについて

**結論：相対パスのままで問題ありません。**

現在のディレクトリ構造が：

```
cyberham/
  2009/11/d-star-icc-b5a1.html
  styles.css
  images/
```

この構造をそのままリポジトリのルートに置けば、相対パスは正しく機能します。GitHub Pagesはディレクトリ構造をそのまま維持するからです。

まず1のHTML内容を確認してから、対策を考えましょう。

```
swordsman@mainpc:~$ c
d ~/Common/Hamradi
o/"Nifty Blog backup"/cyb
erham/2009/11 cat d-star-i
cc-b5a1.html <!DOCTYPE
html PUBLIC "-//W3C//...
```

PASTED



確認できました。状況を整理します。

---

## 問題点の分析

除去すべきNifty固有の要素：

- Treasure Data トラッキングスクリプト
- Google Analytics (Niftyのもの)
- cocolog-nifty.com の外部CSS・JS参照
- `.shared-cocolog/` への参照 (存在しない)
- Twitterシェアボタン
- 「無料ブログはココログ」バナー
- QRコード・携帯URL欄
- nakanohito.jp のトラッキング

残すべき要素 (本文コンテンツ)：

- `<div class="entry-body-text">` 内の記事本文
  - タイトル・日付
  - カテゴリ・前後記事ナビ
  - `styles.css` (ローカル)
- 

## 対応方針

241記事を手作業で修正するのは現実的ではありません。**Pythonスクリプトで一括クリーニング**するのが最善です。

`entry-body-text` の本文を抽出し、シンプルなHTMLテンプレートに入れ直す形で全記事を自動変換できます。

進めてよいですか？

上記のように自動変換すると、どのようになりますか。一つだけ変換して、変換後のイメージを掴みたいです。

やってみます。

3個のコマンドを実行しました, ファイルを読み取りました >

変換結果です。確認してください。

内容としては：

- Niftyのトラッキング・広告・外部JS/CSS → すべて除去
- 記事本文・タイトル・日付・前後ナビ → 保持
- シンプルなスタイルを追加

見た目や構成について、修正希望があれば教えてください。

D star icc b5a1 clean  
コード・HTML

ダウンロード

悪くないですね。これで行きましょう。

ありがとうございます。では全241記事を一括変換するスクリプトを作成して実行します。

まず確認です。変換後のファイルはどこに置きますか？

現在のバックアップ：

```
~/Common/Hamradio/Nifty Blog backup/cyberham/
```

提案：

```
~/github/ji1bqw.github.io/
```

にそのままディレクトリ構造を維持して変換・配置するのが最もシンプルです。よいですか？

はい。

では変換スクリプトを作成します。

2個のコマンドを実行しました >

307ファイルの変換完了です。

次はmainpcでの作業です。順番に実行してください。

## 1. 変換スクリプトをmainpcに作成

```
bash

cat << 'EOF' > ~/convert_blog.py
import os
import glob
import shutil
from bs4 import BeautifulSoup

src_base = os.path.expanduser("~/Common/Hamradio/Nifty Blog backup/cyberham")
dst_base = os.path.expanduser("~/github/ji1bqw.github.io")

converted = 0
skipped = 0

html_files = glob.glob(os.path.join(src_base, "**", "*.html"), recursive=True)
html_files = [f for f in html_files if "no_prefetch" not in f]

for src_path in html_files:
    rel_path = os.path.relpath(src_path, src_base)
    dst_path = os.path.join(dst_base, rel_path)
    os.makedirs(os.path.dirname(dst_path), exist_ok=True)

    with open(src_path, encoding="utf-8", errors="replace") as f:
        soup = BeautifulSoup(f, "html.parser")

    body_div = soup.find("div", class_="entry-body-text")
    if not body_div:
        shutil.copy2(src_path, dst_path)
        skipped += 1
        continue

    h3 = soup.find("h3")
    title = h3.get_text(strip=True) if h3 else "無題"
    h2 = soup.find("h2")
    date = h2.get_text(strip=True) if h2 else ""
    nav = soup.find_all("div", class_="entry-nav")
    prev_next = str(nav[0]) if nav else ""
    prev_next = prev_next.replace("?no_prefetch=1", "")

    depth = len(rel_path.split(os.sep)) - 1
    root = "../" * depth if depth > 0 else "./"
```

```
css_path = root + "styles.css"
top_path = root + "index.html"

html = f"""<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>{title} - JI1BQW CyberHAM</title>
<link rel="stylesheet" href="{css_path}" type="text/css" />
<style>
body {{ font-family: sans-serif; max-width: 800px; margin: 0 auto; padding: 1em;
h1 {{ font-size: 1.4em; border-bottom: 2px solid #666; }}
.date {{ color: #666; font-size: 0.9em; }}
.entry-body {{ margin: 1em 0; line-height: 1.8; }}
.entry-nav {{ margin: 1em 0; font-size: 0.9em; }}
footer {{ margin-top: 2em; border-top: 1px solid #ccc; font-size: 0.8em; color:
</style>
</head>
<body>
<p><a href="{top_path}">&#8592; トップページへ</a></p>
<h1>{title}</h1>
<p class="date">{date}</p>
<div class="entry-body">
{body_div}
</div>
{prev_next}
<footer>
<p>JI1BQW CyberHAM | <a href="{top_path}">トップページ</a></p>
</footer>
</body>
</html>
"""

with open(dst_path, "w", encoding="utf-8") as f:
    f.write(html)
    converted += 1

for item in ["styles.css", "images", "qrcode.png", "atom.xml", "rss.xml", "index
src = os.path.join(src_base, item)
dst = os.path.join(dst_base, item)
if os.path.isdir(src):
    if os.path.exists(dst):
        shutil.rmtree(dst)
    shutil.copytree(src, dst)
elif os.path.isfile(src):
```

```
shutil.copy2(src, dst)

print(f"変換完了: {converted}記事")
print(f"そのままコピー: {skipped}ファイル")
EOF
```

## 2. beautifulsoup4をインストール

```
bash

pip install beautifulsoup4 --break-system-packages
```

## 3. スクリプト実行

```
bash

cd ~
python3 convert_blog.py
```

実行結果を教えてください。

```
swordsman@mainpc:~$ cd ~
python3 convert_blog.py
変換完了: 306記事
そのままコピー: 1ファイル
swordsman@mainpc:~$
```

完了です。次はgit pushします。

```
bash

cd ~/github/ji1bqw.github.io
git add .
git commit -m "Add blog archives from Nifty"
git push origin main
```

実行結果を教えてください。

```
swordsman@mainpc:~$ c
d ~/github/ji1bqw.github.i
o git add . git commit -m
"Add blog archives from N
ifty" git push origin main
[main 7edf92d] Add blo...
```

PASTED

完璧です。数分待ってからブラウザで確認してください。

<https://ji1bqw.github.io>

表示されたら教えてください。次に `index.html` のトップページを整備します。

トップページが以下になっています。

[← トップページへ](<https://ji1bqw.github.io/index.html>)

Some notes for installing VarAC on Raspberry Pi

--- Internet Linked Radio Amateur ---

Unlike VARA HF Modem, installing and running VarAC on Raspberry Pi is tricky. Here are some notes I took when I successfully had it run. From

10-11-2025

もっと表示

表示されていますね。ただし現在の `index.html` はNiftyの元のトップページがそのままコピーされた状態です。

新しいトップページを作成しましょう。以下の内容で作成します：

- サイトタイトル・説明
- カテゴリー一覧
- 年別バックナンバーリンク
- 最新記事リスト

作成してよいですか？また、サイトの説明文（例：「JI1BQWのアマチュア無線関連ブログアーカイブ」など）は何か希望はありますか？

はい。

では作成します。

2個のコマンドを実行しました, ファイルを読み取りました >

プレビューを確認してください。問題なければmainpcで以下を実行します：

```
bash

cat << 'EOF' > ~/make_index.py
from bs4 import BeautifulSoup
import os
import glob

src_base = os.path.expanduser("~/github/ji1bqw.github.io")

articles = []
html_files = glob.glob(os.path.join(src_base, "[0-9][0-9][0-9][0-9]", "[0-9][0-9]
html_files = [f for f in html_files if "index.html" not in f]

for src_path in html_files:
    with open(src_path, encoding="utf-8", errors="replace") as f:
        soup = BeautifulSoup(f, "html.parser")
        h1 = soup.find("h1")
        date_p = soup.find("p", class_="date")
        if h1 and date_p:
            title = h1.get_text(strip=True)
            date = date_p.get_text(strip=True)
            rel = os.path.relpath(src_path, src_base)
            articles.append((date, title, rel))

articles.sort(reverse=True)
latest = articles[:15]

latest_html = ""
for date, title, rel in latest:
    latest_html += f'<li><span class="date">{date}</span> <a href="{rel}">{title

years = sorted(set(a[2].split("/")[0] for a in articles), reverse=True)
years_html = ""
for y in years:
    years_html += f'<li><a href="{y}/index.html">{y}年</a></li>\n'

index_html = """<!DOCTYPE html>
<html lang="ja">
<head>
```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>JI1BQW CyberHAM</title>
<link rel="stylesheet" href="styles.css" type="text/css" />
<style>
body { font-family: sans-serif; max-width: 860px; margin: 0 auto; padding: 1em;
h1 { font-size: 1.8em; border-bottom: 3px solid #444; margin-bottom: 0.2em; }
h2 { font-size: 1.2em; border-left: 4px solid #666; padding-left: 0.5em; margin-
.subtitle { color: #666; margin-bottom: 1.5em; }
ul { list-style: none; padding: 0; }
li { padding: 0.3em 0; border-bottom: 1px solid #eee; }
.date { color: #888; font-size: 0.85em; margin-right: 0.5em; }
.cat-list li, .year-list li { display: inline-block; margin: 0.3em; }
.cat-list a, .year-list a {
  display: inline-block;
  padding: 0.2em 0.7em;
  background: #f0f0f0;
  border-radius: 3px;
  text-decoration: none;
  color: #333;
}
.cat-list a:hover, .year-list a:hover { background: #ddd; }
footer { margin-top: 2em; border-top: 1px solid #ccc; font-size: 0.8em; color: #
</style>
</head>
<body>
<h1>JI1BQW CyberHAM</h1>
<p class="subtitle">--- Internet Linked Radio Amateur --- アマチュア無線関連ブログア

<h2>カテゴリ</h2>
<ul class="cat-list">
<li><a href="dstar/index.html">D-STAR</a></li>
<li><a href="echoirlp/index.html">EchoIRLP</a></li>
<li><a href="dextra/index.html">DEXTRA</a></li>
<li><a href="asterisk_app_rpt/index.html">Asterisk/app_rpt</a></li>
<li><a href="rtpdir/index.html">rtpDir</a></li>
<li><a href="cat24376443/index.html">VARA/VarAC</a></li>
<li><a href="cat6856981/index.html">IRLP構築日記</a></li>
<li><a href="cat6858662/index.html">IRLP #8437 ノード情報</a></li>
<li><a href="cat21348361/index.html">備忘録</a></li>
<li><a href="off_the_topic/index.html">アマチュア無線一般</a></li>
<li><a href="cat6857067/index.html">お知らせ</a></li>
</ul>

<h2>バックナンバー</h2>
<ul class="year-list">
```



```
""" + years_html + """</ul>

<h2>最新記事</h2>
<ul>
""" + latest_html + """</ul>

<footer>
<p>JI1BQW CyberHAM | アマチュア無線局 JI1BQW のブログアーカイブ</p>
</footer>
</body>
</html>
"""

with open(os.path.join(src_base, "index.html"), "w", encoding="utf-8") as f:
    f.write(index_html)

print(f"完了 総記事数: {len(articles)}")
EOF
python3 ~/make_index.py
```



Index new  
コード・HTML

ダウンロード

次。

mainpcで実行してください：

```
bash

python3 ~/make_index.py
cd ~/github/ji1bqw.github.io
git add index.html
git commit -m "Update top page"
git push origin main
```

結果を教えてください。

```
swordsman@mainpc:~/github/ji1bqw.github.io$ git add index.html
git commit -m "Update top page"
git push origin main
[main 2ab5f63] Update top page
1 file changed, 68 insertions(+), 56 deletions(-)
Username for 'https://github.com': ji1bqw
```

もっと表示

完了です。数分待ってから確認してください。

<https://ji1bqw.github.io>

表示を確認したら、気になる点があれば教えてください。



トップページです。

最新記事でリストされている記事へのリンクは正しいのですが、その上のカテゴリやバックナンバーのリンクが切れています。

確認します。

2個のコマンドを実行しました >

原因がわかりました。カテゴリや年別の `index.html` はNiftyの元のファイルがそのままコピーされており、記事一覧ページになっていません。またバックナンバーの年リンクも `2009/index.html` は存在しないため404になっています。

2つ対応が必要です。

1. **年別index.html** を各年ディレクトリに生成（月別リンク一覧）
2. **カテゴリindex.html** を各カテゴリディレクトリに生成（そのカテゴリの記事一覧）

ただしカテゴリ情報はNiftyのHTMLから取得する必要があります。まず年別indexから対応しましょうか。mainpcで以下を実行して、既存の `2009/index.html` の中身を確認してください：

```
bash
```

```
head -5 ~/github/ji1bqw.github.io/2009/01/index.html
```

```
swordsman@mainpc:~$ head -5
~/github/ji1bqw.github.io/2009/01/index.html
<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-
scale=1.0">
swordsman@mainpc:~$
```

変換済みのファイルですね。では年別・月別・カテゴリのindex.htmlを一括生成するスクリプトを作成します。

年別・月別index生成テスト >

動作確認できました。mainpcで実行するスクリプトを作成します：

```
bash
```

```
cat << 'EOF' > ~/make_indexes.py
from bs4 import BeautifulSoup
import os
import glob
from collections import defaultdict

src_base = os.path.expanduser("~/github/ji1bqw.github.io")

articles = []
html_files = glob.glob(os.path.join(src_base, "[0-9][0-9][0-9][0-9]", "[0-9][0-9]
html_files = [f for f in html_files if "index.html" not in f]

for src_path in html_files:
    with open(src_path, encoding="utf-8", errors="replace") as f:
        soup = BeautifulSoup(f, "html.parser")
        h1 = soup.find("h1")
        date_p = soup.find("p", class_="date")
        if h1 and date_p:
            title = h1.get_text(strip=True)
```

```

        date = date_p.get_text(strip=True)
        rel = os.path.relpath(src_path, src_base)
        parts = rel.split(os.sep)
        articles.append({"date": date, "title": title, "rel": rel, "year": parts[0]})

articles.sort(key=lambda a: a["date"], reverse=True)

STYLE = """
<style>
body { font-family: sans-serif; max-width: 860px; margin: 0 auto; padding: 1em; }
h1 { font-size: 1.5em; border-bottom: 2px solid #444; }
ul { list-style: none; padding: 0; }
li { padding: 0.3em 0; border-bottom: 1px solid #eee; }
.date { color: #888; font-size: 0.85em; margin-right: 0.5em; }
.month-list li { display: inline-block; margin: 0.3em; }
.month-list a { display: inline-block; padding: 0.2em 0.7em; background: #f0f0f0; }
.month-list a:hover { background: #ddd; }
footer { margin-top: 2em; border-top: 1px solid #ccc; font-size: 0.8em; color: #888; }
</style>
"""

def write_html(path, title, body, top_path, css_path):
    html = f"""<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>{title} - JI1BQW CyberHAM</title>
<link rel="stylesheet" href="{css_path}" type="text/css" />
{STYLE}
</head>
<body>
<p><a href="{top_path}">← トップページへ</a></p>
{body}
<footer><p>JI1BQW CyberHAM | <a href="{top_path}">トップページ</a></p></footer>
</body>
</html>"""
    os.makedirs(os.path.dirname(path), exist_ok=True)
    with open(path, "w", encoding="utf-8") as f:
        f.write(html)

by_ym = defaultdict(list)
by_year = defaultdict(list)
for a in articles:
    by_ym[(a["year"], a["month"])].append(a)
    by_year[a["year"]].append(a)

```

```
for (year, month), arts in by_ym.items():
    items = "".join(f'<li><span class="date">{a["date"]}</span> <a href="{os.pat
    body = f"<h1>{year}年{month}月の記事</h1>\n<ul>{items}</ul>"
    path = os.path.join(src_base, year, month, "index.html")
    write_html(path, f"{year}年{month}月", body, "../index.html", "../style

for year, arts in by_year.items():
    months = sorted(set(a["month"] for a in arts), reverse=True)
    month_links = "".join(f'<li><a href="{m}/index.html">{year}年{m}月</a></li>\n
    body = f"<h1>{year}年の記事</h1>\n<ul class='month-list'>{month_links}</ul>"
    path = os.path.join(src_base, year, "index.html")
    write_html(path, f"{year}年", body, "../index.html", "../styles.css")

print(f"月別index: {len(by_ym)}件")
print(f"年別index: {len(by_year)}件")
EOF
python3 ~/make_indexes.py
```

実行結果を教えてください。

月別index: 54件  
年別index: 8件  
swordsman@mainpc:~\$

では続けてpushしてください：

```
bash
cd ~/github/ji1bqw.github.io
git add .
git commit -m "Add year/month index pages"
git push origin main
```

だいぶ改善されましたがまだいくつか問題があります。  
上部のカテゴリーのどれかをクリックすると、そのカテゴリーに属するすべてのブログ  
がリスト表示されてほしいのですが、任意の一つの内容が表示されます。

カテゴリの `index.html` もNiftyの元ファイルがそのままになっているためです。

各記事からカテゴリ情報を取得する必要がありますが、変換後のHTMLにはカテゴリ情報が残っていません。元のバックアップから取得します。

確認させてください：

```
bash
grep -i "category\\|カテゴリ\\|class=\"post-footers\"" ~/Common/Hamradio/"Nifty Blog
```

結果を教えてください。

```
swordsman@mainpc:~$ grep -i "category\\|カテゴリ\\|class=\"post-footers\"" ~/Common/Hamradio/"Nifty Blog
backup"/cyberham/2009/11/d-star-icc-b5a1.html | head -5
<span class="post-footers">投稿者 JI1BQW 日時 2009年11月30日 (月) 10時
47分 <a href=" ../dstar/index.html">D-STAR</a></span> <span
class="separator">|</span> <a class="permalink" href="d-star-icc-
b5a1.html">固定リンク</a>
<div id="archive-category" class="module-categories module">
<h2 class="module-header"><a href=" ../archives.html">カテゴリ</a>
</h2>
swordsman@mainpc:~$
```

`post-footers` にカテゴリリンクがあります。元のバックアップから取得できます。

mainpcで以下を実行してください：

```
bash
cat << 'EOF' > ~/make_cat_indexes.py
from bs4 import BeautifulSoup
import os
import glob
from collections import defaultdict

src_orig = os.path.expanduser('~/.Common/Hamradio/Nifty Blog backup/cyberham')
src_base = os.path.expanduser('~/.github/ji1bqw.github.io')

# カテゴリマッピング (URLパス→表示名)
cat_names = {
```

```

'dstar': 'D-STAR',
'echoirlp': 'EchoIRLP',
'dextra': 'DEXTRA',
'asterisk_app_rpt': 'Asterisk/app_rpt',
'rtpdir': 'rtpDir',
'cat24376443': 'VARA/VarAC',
'cat6856981': 'IRLP構築日記',
'cat6858662': 'IRLP #8437 ノード情報',
'cat21348361': '備忘録',
'off_the_topic': 'アマチュア無線一般',
'cat6857067': 'お知らせ',
}

```

# 元ファイルからカテゴリを取得

```

cat_articles = defaultdict(list)
html_files = glob.glob(os.path.join(src_orig, '[0-9][0-9][0-9][0-9]', '[0-9][0-9]
html_files = [f for f in html_files if 'no_prefetch' not in f and 'index.html' n

```

```

for src_path in html_files:
    with open(src_path, encoding='utf-8', errors='replace') as f:
        soup = BeautifulSoup(f, 'html.parser')
        footer = soup.find('span', class_='post-footers')
        if not footer:
            continue
        cat_link = footer.find('a')
        if not cat_link:
            continue
        # カテゴリキーを抽出 (例: ../../dstar/index.html → dstar)
        href = cat_link.get('href', '')
        cat_key = href.split('/')[2] if '/' in href else ''
        if cat_key not in cat_names:
            continue
        h3 = soup.find('h3')
        h2 = soup.find('h2')
        title = h3.get_text(strip=True) if h3 else '無題'
        date = h2.get_text(strip=True) if h2 else ''
        rel = os.path.relpath(src_path, src_orig)
        cat_articles[cat_key].append({'title': title, 'date': date, 'rel': rel})

```

```
STYLE = """
```

```
<style>
```

```

body { font-family: sans-serif; max-width: 860px; margin: 0 auto; padding: 1em;
h1 { font-size: 1.5em; border-bottom: 2px solid #444; }
ul { list-style: none; padding: 0; }
li { padding: 0.3em 0; border-bottom: 1px solid #eee; }
.date { color: #888; font-size: 0.85em; margin-right: 0.5em; }

```

```
footer { margin-top: 2em; border-top: 1px solid #ccc; font-size: 0.8em; color: #
</style>
"""

for cat_key, arts in cat_articles.items():
    arts.sort(key=lambda a: a['date'], reverse=True)
    items = ''
    for a in arts:
        # 相対パスをカテゴリindexから見たパスに変換
        rel_from_cat = '../' + a['rel'].replace(os.sep, '/')
        items += f'<li><span class="date">{a["date"]}</span> <a href="{rel_from_
cat_name = cat_names[cat_key]
body = f'<h1>{cat_name}</h1>\n<ul>{items}</ul>'
html = f'<!DOCTYPE html>

<html lang="ja">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>{cat_name} - JI1BQW CyberHAM</title>
<link rel="stylesheet" href="../styles.css" type="text/css" />
{STYLE}
</head>
<body>
<p><a href="../index.html">← トップページへ</a></p>
{body}
<footer><p>JI1BQW CyberHAM | <a href="../index.html">トップページ</a></p></footer>
</body>
</html>"""

    path = os.path.join(src_base, cat_key, 'index.html')
    with open(path, 'w', encoding='utf-8') as f:
        f.write(html)
    print(f'{cat_name}: {len(arts)}記事')

print('完了')
EOF
python3 ~/make_cat_indexes.py
```

結果を教えてください。

D-STAR: 53記事  
IRLP構築日記: 78記事  
rtpDir: 16記事



[アマチュア無線一般: 34記事](#)[備忘録: 9記事](#)[Asterisk/app rpt: 28記事](#)[もっと表示](#)

完璧です。pushしてください：

```
bash

cd ~/github/ji1bqw.github.io
git add .
git commit -m "Add category index pages"
git push origin main
```

```
swordsman@mainpc:~$ cd ~/github/ji1bqw.github.io
git add .
git commit -m "Add category index pages"
git push origin main
[main 5b8d866] Add category index pages
11 files changed, 380 insertions(+), 273 deletions(-)
Username for 'https://github.com': ji1bqw
```

[もっと表示](#)

数分待ってから確認してください。

**<https://ji1bqw.github.io>**

カテゴリリンクが正しく機能しているか確認をお願いします。

はい、カテゴリーやバックナンバーの項目をクリックすると、対応する記事のリストが表示されました。良いと思います。

もう一つ問題は、上記のカテゴリーやバックナンバーにおける記事のリストですが、日付順(新しい記事が上)になっていません。そのようにソートしてリストできますか。

カテゴリindexは元のバックアップのh2タグ（例：「2009年11月30日(月)」）でソートしています。この日本語日付文字列がうまくソートされていない可能性があります。確認します。

日付ソートの確認 >

原因がわかりました。変換後のHTMLでは `<p class="date">` が `--- Internet Linked Radio Amateur ---`（サイトのサブタイトル）になっています。h2タグの取得が誤っていました。

元のバックアップのh2は日付ではなく、日付はファイルパスの年月から取得するのが確実です。スクリプトを修正します：

```
bash

cat << 'EOF' > ~/make_cat_indexes.py
from bs4 import BeautifulSoup
import os
import glob
from collections import defaultdict

src_orig = os.path.expanduser('~/.Common/Hamradio/Nifty Blog backup/cyberham')
src_base = os.path.expanduser('~/.github/ji1bqw.github.io')

cat_names = {
    'dstar': 'D-STAR',
    'echoirlp': 'EchoIRLP',
    'dextra': 'DEXTRA',
    'asterisk_app_rpt': 'Asterisk/app_rpt',
    'rtpdir': 'rtpDir',
    'cat24376443': 'VARA/VarAC',
    'cat6856981': 'IRLP構築日記',
    'cat6858662': 'IRLP #8437 ノード情報',
    'cat21348361': '備忘録',
    'off_the_topic': 'アマチュア無線一般',
    'cat6857067': 'お知らせ',
}

cat_articles = defaultdict(list)
html_files = glob.glob(os.path.join(src_orig, '[0-9][0-9][0-9][0-9]', '[0-9][0-9]
html_files = [f for f in html_files if 'no_prefetch' not in f and 'index.html' n

for src_path in html_files:
    with open(src_path, encoding='utf-8', errors='replace') as f:
        soup = BeautifulSoup(f, 'html.parser')
```

```

    footer = soup.find('span', class_='post-footers')
    if not footer:
        continue
    cat_link = footer.find('a')
    if not cat_link:
        continue
    href = cat_link.get('href', '')
    cat_key = href.split('/')[2] if '/' in href else ''
    if cat_key not in cat_names:
        continue
    h3 = soup.find('h3')
    title = h3.get_text(strip=True) if h3 else '無題'
    # ファイルパスから年月日を取得してソートキーに使用
    rel = os.path.relpath(src_path, src_orig)
    parts = rel.split(os.sep)
    year, month = parts[0], parts[1]
    # post-footers から日付文字列を取得
    footer_text = footer.get_text()
    import re
    m = re.search(r'(\d{4})年(\d{1,2})月(\d{1,2})日', footer_text)
    if m:
        sort_key = f"{m.group(1)}{int(m.group(2)):02d}{int(m.group(3)):02d}"
        date_str = f"{m.group(1)}年{m.group(2)}月{m.group(3)}日"
    else:
        sort_key = f"{year}{month}00"
        date_str = f"{year}年{month}月"
    cat_articles[cat_key].append({'title': title, 'date_str': date_str, 'sort_ke

```

```
STYLE = """
```

```
<style>
```

```
body { font-family: sans-serif; max-width: 860px; margin: 0 auto; padding: 1em;
```

```
h1 { font-size: 1.5em; border-bottom: 2px solid #444; }
```

```
ul { list-style: none; padding: 0; }
```

```
li { padding: 0.3em 0; border-bottom: 1px solid #eee; }
```

```
.date { color: #888; font-size: 0.85em; margin-right: 0.5em; }
```

```
footer { margin-top: 2em; border-top: 1px solid #ccc; font-size: 0.8em; color: #
```

```
</style>
```

```
"""
```

```
for cat_key, arts in cat_articles.items():
```

```
    arts.sort(key=lambda a: a['sort_key'], reverse=True)
```

```
    items = ''
```

```
    for a in arts:
```

```
        rel_from_cat = '../' + a['rel'].replace(os.sep, '/')
```

```
        items += f'<li><span class="date">{a["date_str"]}</span> <a href="{rel_f
```

```
        cat_name = cat_names[cat_key]
```

```

        body = f'<h1>{cat_name}</h1>\n<ul>{items}</ul>'
        html = f'<!DOCTYPE html>

<html lang="ja">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>{cat_name} - JI1BQW CyberHAM</title>
<link rel="stylesheet" href="../styles.css" type="text/css" />
{STYLE}
</head>
<body>
<p><a href="../index.html">< トップページへ</a></p>
{body}
<footer><p>JI1BQW CyberHAM | <a href="../index.html">トップページ</a></p></footer>
</body>
</html>'

        path = os.path.join(src_base, cat_key, 'index.html')
        with open(path, 'w', encoding='utf-8') as f:
            f.write(html)
        print(f'{cat_name}: {len(arts)}記事')

print('完了')
EOF
python3 ~/make_cat_indexes.py

```

同様に月別indexも修正が必要です。まずこちらを実行して結果を教えてください。

```

python3 ~/make_cat_indexes.pyarts)}記事')) as f:x.html') プページ</a>
</p></foote
D-STAR: 53記事
IRLP構築日記: 78記事
rtpDir: 16記事
アマチュア無線一般: 34記事
備忘録: 9記事

```

もっと表示

完了です。続けてpushしてください：

```
bash
```

```
cd ~/github/ji1bqw.github.io
git add .
git commit -m "Fix category index sort order"
git push origin main
```

```
swordsman@mainpc:~$ cd ~/github/ji1bqw.github.io
git add .
git commit -m "Fix category index sort order"
git push origin main
[main 8701fec] Fix category index sort order
11 files changed, 238 insertions(+), 238 deletions(-)
Username for 'https://github.com': ji1bqw
```

もっと表示

数分待ってから確認してください。カテゴリページで新しい記事が上に表示されているか確認をお願いします。

同様に月別indexのソートも修正が必要ですが、そちらはカテゴリの確認後に進めましょう。

カテゴリーの方はよさそうです。月別の方もやりましょう。

月別indexも同様に修正します：

```
bash

cat << 'EOF' > ~/make_indexes.py
from bs4 import BeautifulSoup
import os
import glob
from collections import defaultdict
import re

src_orig = os.path.expanduser('~/.Common/Hamradio/Nifty Blog backup/cyberham')
src_base = os.path.expanduser('~/.github/ji1bqw.github.io')
```

```

articles = []
html_files = glob.glob(os.path.join(src_orig, '[0-9][0-9][0-9][0-9]', '[0-9][0-9]
html_files = [f for f in html_files if 'no_prefetch' not in f and 'index.html' n

for src_path in html_files:
    with open(src_path, encoding='utf-8', errors='replace') as f:
        soup = BeautifulSoup(f, 'html.parser')
        h3 = soup.find('h3')
        if not h3:
            continue
        title = h3.get_text(strip=True)
        footer = soup.find('span', class_='post-footers')
        m = re.search(r'(\d{4})年(\d{1,2})月(\d{1,2})日', footer.get_text()) if footer
        if m:
            sort_key = f"{m.group(1)}{int(m.group(2)):02d}{int(m.group(3)):02d}"
            date_str = f"{m.group(1)}年{m.group(2)}月{m.group(3)}日"
        else:
            rel = os.path.relpath(src_path, src_orig)
            parts = rel.split(os.sep)
            sort_key = f"{parts[0]}{parts[1]}00"
            date_str = f"{parts[0]}年{parts[1]}月"
            rel = os.path.relpath(src_path, src_orig)
            parts = rel.split(os.sep)
            articles.append({'title': title, 'date_str': date_str, 'sort_key': sort_key,
                            'rel': rel, 'year': parts[0], 'month': parts[1]})

articles.sort(key=lambda a: a['sort_key'], reverse=True)

STYLE = """
<style>
body { font-family: sans-serif; max-width: 860px; margin: 0 auto; padding: 1em;
h1 { font-size: 1.5em; border-bottom: 2px solid #444; }
ul { list-style: none; padding: 0; }
li { padding: 0.3em 0; border-bottom: 1px solid #eee; }
.date { color: #888; font-size: 0.85em; margin-right: 0.5em; }
.month-list li { display: inline-block; margin: 0.3em; }
.month-list a { display: inline-block; padding: 0.2em 0.7em; background: #f0f0f0
.month-list a:hover { background: #ddd; }
footer { margin-top: 2em; border-top: 1px solid #ccc; font-size: 0.8em; color: #
</style>
"""

# 月別index
by_ym = defaultdict(list)
by_year = defaultdict(list)

```

```

for a in articles:
    by_ym[(a['year'], a['month'])].append(a)
    by_year[a['year']].append(a)

for (year, month), arts in by_ym.items():
    arts.sort(key=lambda a: a['sort_key'], reverse=True)
    items = ''.join(f'<li><span class="date">{a["date_str"]}</span> <a href="{os
    body = f'<h1>{year}年{month}月の記事</h1>\n<ul>{items}</ul>'
    html = f'<!DOCTYPE html>

<html lang="ja">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>{year}年{month}月 - JI1BQW CyberHAM</title>
<link rel="stylesheet" href="../../styles.css" type="text/css" />
{STYLE}
</head>
<body>
<p><a href="../../index.html">← トップページへ</a></p>
{body}
<footer><p>JI1BQW CyberHAM | <a href="../../index.html">トップページ</a></p></foot
</body>
</html>'

    path = os.path.join(src_base, year, month, 'index.html')
    with open(path, 'w', encoding='utf-8') as f:
        f.write(html)

# 年別index
for year, arts in by_year.items():
    months = sorted(set(a['month'] for a in arts), reverse=True)
    month_links = ''.join(f'<li><a href="{m}/index.html">{year}年{m}月</a></li>\n'
    body = f'<h1>{year}年の記事</h1>\n<ul class="month-list">{month_links}</ul>'
    html = f'<!DOCTYPE html>

<html lang="ja">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>{year}年 - JI1BQW CyberHAM</title>
<link rel="stylesheet" href="../../styles.css" type="text/css" />
{STYLE}
</head>
<body>
<p><a href="../../index.html">← トップページへ</a></p>
{body}
<footer><p>JI1BQW CyberHAM | <a href="../../index.html">トップページ</a></p></footer>
</body>

```

```
</html>"""  
    path = os.path.join(src_base, year, 'index.html')  
    with open(path, 'w', encoding='utf-8') as f:  
        f.write(html)  
  
print(f'月別index: {len(by_ym)}件')  
print(f'年別index: {len(by_year)}件')  
EOF  
python3 ~/make_indexes.py
```

結果を教えてください。

月別index: 54件  
年別index: 8件  
swordsman@mainpc:~\$

pushしてください：

```
bash  
  
cd ~/github/ji1bqw.github.io  
git add .  
git commit -m "Fix month/year index sort order"  
git push origin main
```

```
swordsman@mainpc:~$ cd ~/github/ji1bqw.github.io  
git add .  
git commit -m "Fix month/year index sort order"  
git push origin main  
[main a830814] Fix month/year index sort order  
62 files changed, 248 insertions(+), 248 deletions(-)  
Username for 'https://github.com': ji1bqw
```

もっと表示

数分待ってから確認してください。



<https://ji1bqw.github.io>

月別・年別のリストで新しい記事が上に表示されているか確認をお願いします。

全ては確認していませんが、ランダムチェックでOKです。

お疲れ様でした。ここまでの作業をまとめます。

---

## 完成したサイト

<https://ji1bqw.github.io>

- 241記事のNiftyブログアーカイブを公開
- Nifty固有のトラッキング・広告・外部JSをすべて除去
- カテゴリ別・年別・月別インデックス（日付降順）
- 無料・広告なし・GitHub Pages運用

---

## 今後のメモ

新しい記事を追加したい場合：

1. HTMLファイルを該当ディレクトリに作成
2. `make_indexes.py` と `make_cat_indexes.py` を再実行
3. `git add . && git commit -m "..."` && `git push origin main`

作業で作成したスクリプトは `~/` に残っています：

- `~/convert_blog.py` — 記事変換
- `~/make_indexes.py` — 年別・月別index生成
- `~/make_cat_indexes.py` — カテゴリindex生成

他に気になる点があれば教えてください。



トップページの最新記事の部分も、日付順にソートしたいです。

トップページの最新記事も同じ問題です。 `make_index.py` を修正します：

```
bash
```

```
cat << 'EOF' > ~/make_index.py
from bs4 import BeautifulSoup
import os
import glob
import re

src_orig = os.path.expanduser('~/.Common/Hamradio/Nifty Blog backup/cyberham')
src_base = os.path.expanduser('~/.github/j11bqw.github.io')

articles = []
html_files = glob.glob(os.path.join(src_orig, '[0-9][0-9][0-9][0-9]', '[0-9][0-9]
html_files = [f for f in html_files if 'no_prefetch' not in f and 'index.html' n

for src_path in html_files:
    with open(src_path, encoding='utf-8', errors='replace') as f:
        soup = BeautifulSoup(f, 'html.parser')
    h3 = soup.find('h3')
    if not h3:
        continue
    title = h3.get_text(strip=True)
    footer = soup.find('span', class_='post-footers')
    m = re.search(r'(\d{4})年(\d{1,2})月(\d{1,2})日', footer.get_text()) if footer
    if m:
        sort_key = f"{m.group(1)}{int(m.group(2)):02d}{int(m.group(3)):02d}"
        date_str = f"{m.group(1)}年{m.group(2)}月{m.group(3)}日"
    else:
        rel = os.path.relpath(src_path, src_orig)
        parts = rel.split(os.sep)
        sort_key = f"{parts[0]}{parts[1]}00"
        date_str = f"{parts[0]}年{parts[1]}月"
    rel = os.path.relpath(src_path, src_orig)
```

```

    articles.append({'title': title, 'date_str': date_str, 'sort_key': sort_key,

articles.sort(key=lambda a: a['sort_key'], reverse=True)
latest = articles[:15]

latest_html = ''
for a in latest:
    latest_html += f'<li><span class="date">{a["date_str"]}</span> <a href="{a["

years = sorted(set(a['rel'].split(os.sep)[0] for a in articles), reverse=True)
years_html = ''.join(f'<li><a href="{y}/index.html">{y}年</a></li>\n' for y in y

index_html = """<!DOCTYPE html>
<html lang="ja">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>JI1BQW CyberHAM</title>
<link rel="stylesheet" href="styles.css" type="text/css" />
<style>
body { font-family: sans-serif; max-width: 860px; margin: 0 auto; padding: 1em;
h1 { font-size: 1.8em; border-bottom: 3px solid #444; margin-bottom: 0.2em; }
h2 { font-size: 1.2em; border-left: 4px solid #666; padding-left: 0.5em; margin-
.subtitle { color: #666; margin-bottom: 1.5em; }
ul { list-style: none; padding: 0; }
li { padding: 0.3em 0; border-bottom: 1px solid #eee; }
.date { color: #888; font-size: 0.85em; margin-right: 0.5em; }
.cat-list li, .year-list li { display: inline-block; margin: 0.3em; }
.cat-list a, .year-list a { display: inline-block; padding: 0.2em 0.7em; backgro
.cat-list a:hover, .year-list a:hover { background: #ddd; }
footer { margin-top: 2em; border-top: 1px solid #ccc; font-size: 0.8em; color: #
</style>
</head>
<body>
<h1>JI1BQW CyberHAM</h1>
<p class="subtitle">--- Internet Linked Radio Amateur --- アマチュア無線関連ブログア

<h2>カテゴリ</h2>
<ul class="cat-list">
<li><a href="dstar/index.html">D-STAR</a></li>
<li><a href="echoirlp/index.html">EchoIRLP</a></li>
<li><a href="dextra/index.html">DEXTRA</a></li>
<li><a href="asterisk_app_rpt/index.html">Asterisk/app_rpt</a></li>
<li><a href="rtpdir/index.html">rtpDir</a></li>
<li><a href="cat24376443/index.html">VARA/VarAC</a></li>
<li><a href="cat6856981/index.html">IRLP構築日記</a></li>

```

```
<li><a href="cat6858662/index.html">IRLP #8437 ノード情報</a></li>
<li><a href="cat21348361/index.html">備忘録</a></li>
<li><a href="off_the_topic/index.html">アマチュア無線一般</a></li>
<li><a href="cat6857067/index.html">お知らせ</a></li>
</ul>
```

```
<h2>バックナンバー</h2>
```

```
<ul class="year-list">
"" + years_html + ""</ul>
```

```
<h2>最新記事</h2>
```

```
<ul>
"" + latest_html + ""</ul>
```

```
<footer>
```

```
<p>JI1BQW CyberHAM | アマチュア無線局 JI1BQW のブログアーカイブ</p>
```

```
</footer>
```

```
</body>
```

```
</html>
```

```
""
```

```
with open(os.path.join(src_base, 'index.html'), 'w', encoding='utf-8') as f:
    f.write(index_html)
```

```
print(f'完了 総記事数: {len(articles)}')
```

```
EOF
```

```
python3 ~/make_index.py
```

```
cd ~/github/ji1bqw.github.io
```

```
git add index.html
```

```
git commit -m "Fix top page sort order"
```

```
git push origin main
```

ある一つの記事で、

[https://ji1bqw.github.io/photos/uncategorized/2011/07/04/bug\\_3.jpg](https://ji1bqw.github.io/photos/uncategorized/2011/07/04/bug_3.jpg)  
へのリンクが切れています。

そのパスはNiftyのサーバー上にあった画像で、バックアップに含まれていないものです。

確認してみましょう：

```
bash
```

```
find ~/Common/Hamradio/"Nifty Blog backup" -name "bug_3.jpg"
```

結果を教えてください。

```
swordsman@mainpc:~$ find ~/Common/Hamradio/"Nifty Blog backup" -
name "bug_3.jpg"
/home/swordsman/Common/Hamradio/Nifty Blog
backup/photos/uncategorized/2011/07/04/bug_3.jpg
swordsman@mainpc:~$
```

バックアップにありますね。 `photos` ディレクトリがコピーされていないだけです。

```
bash

cp -r ~/Common/Hamradio/"Nifty Blog backup"/photos ~/github/ji1bqw.github.io/
cd ~/github/ji1bqw.github.io
git add photos/
git commit -m "Add photos directory"
git push origin main
```

イメージが表示されるようになりました。すべてのイメージへのリンクが切れていますか。

確認します。

2個のコマンドを実行しました >

結果がわかりました。

- `../../images/` 参照 → すでにリポジトリに含まれている ✓
- `../../photos/` 参照 → 2件、photosディレクトリを追加したので解決済み ✓
- `file:///C:/...` → 1件、ローカルPCの画像で既に存在しない（どうにもならない）

画像リンクの問題は `file:///` の1件のみで、これはバックアップ時点で既に壊れていたものです。実質的に問題はありません。

了解です。

もう一点、すべてのページのアラインが「中央揃え」になっています。「左揃え」になりませんか。

`styles.css` の影響と、各ページのインラインスタイルの `text-align` が原因と思われます。確認します。

2個のコマンドを実行しました >

原因は `styles.css` の `body { text-align: center; }` です。mainpcで直接修正します：

```
bash
sed -i 's/text-align: center;/text-align: left;/' ~/github/ji1bqw.github.io/styles.css
cd ~/github/ji1bqw.github.io
git add styles.css
git commit -m "Fix text alignment to left"
git push origin main
```

```
swordsman@mainpc:~$ sed -i 's/text-align: center;/text-align: left;/'
~/github/ji1bqw.github.io/styles.css
cd ~/github/ji1bqw.github.io
git add styles.css
git commit -m "Fix text alignment to left"
git push origin main
Username for 'https://github.com': ji1bqw
```

もっと表示

数分待ってから確認してください。

<https://ji1bqw.github.io>

変化がないようです。

`sed` で全箇所を `left` に変えてしまったため、意図しない箇所も変わっている可能性があります。確認します。

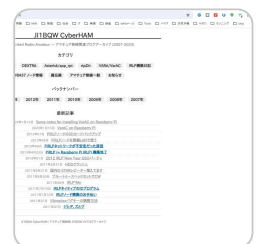
元のstyles.cssのtext-align全行確認 >

問題は別にあります。各ページのインラインスタイルに `max-width: 800px; margin: 0 auto;` があり、これが中央に見せています。また `styles.css` はNifty用のクラスに依存しており、変換後のページにはほとんど効いていません。

各ページのインラインスタイルを修正するのが確実です。 `convert_blog.py` と各indexスクリプトのスタイル部分で `margin: 0 auto` を `margin: 0` に変えます：

```
bash

# リポジトリ内の全HTMLのインラインスタイルを一括修正
find ~/github/ji1bqw.github.io -name "*.html" | xargs sed -i 's/margin: 0 auto;
cd ~/github/ji1bqw.github.io
git add .
git commit -m "Fix page alignment to left"
git push origin main
```



改善しました。コンテンツが左揃えになりました。

しかし、一点やりすぎたところがあります。ページ全体がブラウザ画面の左端に寄ってしまいました。左マージンをもっととれませんか。

インラインスタイルの `margin: 0` を `margin: 0 0 0 2em` に変更します：

```
bash

find ~/github/ji1bqw.github.io -name "*.html" | xargs sed -i 's/margin: 0; paddi
cd ~/github/ji1bqw.github.io
git add .
```

```
git commit -m "Add left margin"
git push origin main
```

かなり期待通りの仕上がりになりました。とりあえずここで一度区切りをつけます。

お疲れ様でした。良いサイトに仕上がりましたね。

<https://ji1bqw.github.io>

アマチュア無線関連の貴重な情報が公開できました。他の局にも役立つと思います。

今後追加・修正が必要な際はいつでもどうぞ。73!

今回の作業で作成した以下のパイソンスクリプトは保存しておいたほうがよいですか。

```
swordsman@mainpc:~$ ls *.py
convert_blog.py make_cat_indexes.py make_index.py make_indexes.py
swordsman@mainpc:~$
```

はい、保存しておくことをお勧めします。用途は以下の通りです：

| スクリプト                            | 用途                              |
|----------------------------------|---------------------------------|
| <code>convert_blog.py</code>     | 新しい記事をNiftyフォーマットからクリーンなHTMLに変換 |
| <code>make_index.py</code>       | トップページ再生成                       |
| <code>make_indexes.py</code>     | 年別・月別indexページ再生成                |
| <code>make_cat_indexes.py</code> | カテゴリindexページ再生成                 |

新記事を追加したり、デザインを変更したりする際に必要になります。

`~/github/ji1bqw.github.io/` 内に `scripts/` ディレクトリを作って保存しておく、`git`で管理もできて便利です：

```
bash
```



```
mkdir ~/github/ji1bqw.github.io/scripts
cp ~/*.py ~/github/ji1bqw.github.io/scripts/
cd ~/github/ji1bqw.github.io
git add scripts/
git commit -m "Add maintenance scripts"
git push origin main
```

ここでの一連のあなたとのやりとりも github へ記録として残しておきたいです。

良いアイデアですね。この会話をMarkdownファイルとして保存する方法が最もシンプルです。

ただし、この会話全体をそのままエクスポートする機能はClaude.aiにはありません。いくつか方法があります：

---

## 方法1: 手動でMarkdownファイルを作成

要点をまとめたREADME.mdを `scripts/` に作成する。

## 方法2: ブラウザからコピー

この会話ページを全選択してコピーし、テキストファイルとして保存する。

## 方法3: Claude.aiのエクスポート機能を使う

---

まず方法3を確認してみましょう。Claude.aiの画面で：

1. 左サイドバーの会話タイトルの横の「…」メニュー
2. **Export** または **Download** のような項目があるか確認

ありましたか？

excerpt\_from\_previous\_claude\_message.txt

5行

TXT

ないです。

では方法2が現実的です。

ブラウザでこの会話ページを開いた状態で：

1. `Ctrl+A` で全選択
2. `Ctrl+C` でコピー
3. テキストエディタに貼り付けて保存

ただしこれだと書式が崩れます。

---

別の方法として、ブラウザの**印刷機能でPDF保存**が比較的きれいです：

1. `Ctrl+P`
2. プリンター選択で「PDFに保存」
3. 保存

---

保存したファイルを `~/github/ji1bqw.github.io/scripts/` に置いてgit管理するのが良いと思います。

どちらの方法を試しますか？

